

AUDITORIA TÉCNICA DO CÓDIGO

Cálculo de Custo, Valor Total e Lucro

Análise do tratamento de preço médio, quantidades fracionadas e rentabilidade da carteira.

Projeto: CompraCerta

Data: 16 de setembro de 2026

Escopo: cálculo de ativos, formulário, cripto fracionada e componentes de exibição

1. Resumo executivo

Conclusão: o sistema trata `pm` como preço médio unitário. O custo total é calculado como `quantidade × preço médio`; o valor atual como `quantidade × preço atual`.

```
valorTotal = qty * price
valorInvestido = qty * pm
lucro = valorTotal - valorInvestido
rentabilidade = (lucro / valorInvestido) * 100
```

Para cripto fracionada, a fórmula matemática está correta. O principal ponto de atenção está no formulário, que usa `step="0.01"` para campos numéricos e pode dificultar quantidades como `0.005`.

2. Custo total

O cálculo está em `src/services/calculationService.js`, na função `calculatePortfolioTotals`:

```
const valorInvestido = Number(item.qty) * Number(item.pm);
```

Isso confirma que o sistema multiplica:

Custo Total = Quantidade × Preço Médio Unitário

O usuário deve informar o preço de uma unidade inteira do ativo, mesmo quando compra uma fração.

Exemplo: 0.005 BTC

```
Quantidade: 0.005 BTC
Preço médio unitário: R$ 300.000,00

Custo total = 0.005 × 300.000
Custo total = R$ 1.500,00
```

Preenchimento incorreto: informar R\$ 1.500,00 como preço médio faria o sistema calcular `0.005 × 1.500 = R$ 7,50`.

3. Valor total atual

O valor de mercado é calculado em `calculationService.js`:

```
const valorTotal = Number(item.qty) * Number(item.price);
```

Com 0.005 BTC e preço atual de R\$ 320.000,00:

```
Valor atual = 0.005 × 320.000
Valor atual = R$ 1.600,00
```

A tabela exibe `item.valorTotal` em `src/components/Portfolio/AssetsTable.jsx`.

4. Lucro e prejuízo

O lucro individual é calculado por:

```
const lucro = valorTotal - valorInvestido;
```

No exemplo:

```
Lucro = R$ 1.600,00 - R$ 1.500,00  
Lucro = R$ 100,00
```

O valor e a cor do lucro são exibidos em `src/components/Portfolio/AssetsTable.jsx`.

5. Rentabilidade percentual

A rentabilidade individual é calculada por:

```
const rentabilidade = valorInvestido > 0  
  ? (lucro / valorInvestido) * 100  
  : 0;
```

Exemplo:

```
Rentabilidade = (100 / 1.500) * 100  
Rentabilidade = 6,67%
```

Para quantidade fracionada, a quantidade aparece no numerador e no denominador e se cancela matematicamente:

```
(qty × preçoAtual - qty × preçoMédio) / (qty × preçoMédio)
```

Portanto, a rentabilidade percentual depende da variação entre o preço unitário atual e o preço unitário médio, não do tamanho da posição.

6. Tratamento no formulário

Os campos comuns estão definidos em `src/components/Forms/AssetForm.jsx`:

```
['qty', 'qtd', ''], ['pm', 'pm', ''], ['price', 'cotacao', '']
```

Os campos numéricos recebem:

```
type="number"  
step="0.01"
```

Atenção paracripto: embora o cálculo aceite `0.005`, `step="0.01"` pode fazer o navegador considerar o valor inválido. Paracripto, recomenda-se `step="any"` ou um passo menor, como `0.00000001`.

7. Conversão e persistência do ativo

Ao salvar, `src/pages/PortfolioPage.jsx` converte os campos:

```
qty: Number(formData.qty) || 1,
pm: parseOptionalNumber(formData.pm),
price: Number(formData.price)
```

Quantidade válida como `0.005` é preservada. Quantidade vazia ou zero cai no padrão `1`.

O preço médio utiliza `parseNumericInput`, que aceita formatos brasileiros como `300.000,50`. Já o preço atual usa `Number()` diretamente, podendo falhar com vírgula decimal em determinados formatos.

8. Totais gerais da carteira

Em `calculationService.js`, os valores são acumulados:

```
patrimonio += valorTotal;
investido += valorInvestido;
```

Depois:

```
const lucroTotal = patrimonio - investido;
const rentabilidadeTotal = investido > 0
  ? (lucroTotal / investido) * 100
  : 0;
```

A função é executada via `useMemo` em `src/App.jsx` e os indicadores são exibidos em `src/components/Common/KPICards.jsx`.

9. Referências e regra de preenchimento

Informação	Arquivo	Regra
Custo total	<code>src/services/calculationService.js</code>	<code>qty × pm</code>
Valor atual	<code>src/services/calculationService.js</code>	<code>qty × price</code>
Lucro	<code>src/services/calculationService.js</code>	<code>valorTotal - valorInvestido</code>
Rentabilidade	<code>src/services/calculationService.js</code>	<code>lucro / valorInvestido × 100</code>
Entrada	<code>src/components/Forms/AssetForm.jsx</code>	Quantidade decimal limitada por <code>step=0.01</code>
Persistência	<code>src/pages/PortfolioPage.jsx</code>	<code>pm</code> e <code>price</code> são valores unitários

Regra para o usuário:

Quantidade = quantidade real adquirida.

Preço Médio = preço unitário de uma unidade inteira.

Preço Atual = preço unitário atual de uma unidade inteira.

Relatório gerado a partir do estado atual do código-fonte do projeto CompraCerta.